



TALLER PRÁCTICO DE JAVASCRIPT BÁSICO

Simulador de Finanzas Personales



Duración 2 horas



Nivel Principiante



Requisitos HTML + CSS básico

Mentor: Senior Fullstack Developer

Proyecto: Simulador de Finanzas Personales interactivo en el navegador

Al final de este taller habrás construido una app real que muestra ingresos, gastos y saldo, con lógica dinámica en JavaScript. Sin frameworks, sin magia: solo JS puro.



Hoja de Ruta — Los 4 Módulos

Módulo	Tiempo	Tema	Entregable
01	0:00 – 0:30 Conceptos	Variables, tipos de datos y funciones	Declarar variables del simulador
02	0:30 – 1:00 Lógica	Condicionales, operadores y bucles	Función calcular saldo
03	1:00 – 1:30 DOM	getElementById, innerHTML, eventos	Pantalla actualiza con botón
04	1:30 – 2:00 Proyecto Final	Integración completa + estilos	App funcional y presentable



Tip pedagógico

Construye el proyecto de forma incremental. Cada módulo agrega una capa. Al final del módulo 4 el código completo habrá nacido de sus propias manos.

Dibuja en la pizarra la "arquitectura" antes de cada módulo: qué piezas ya tenemos y qué vamos a agregar.

MÓDULO 01 — Variables y Funciones (0:00 – 0:30)

Conceptos con Analogías

const y let — Las etiquetas de tus cajas

Imagina que tu cuarto es la memoria del navegador. Tienes cajas donde guardas cosas.

const — Caja sellada con llave	let — Caja normal sin llave
Guardas algo y nadie lo puede cambiar. Ejemplo: tu fecha de nacimiento, el nombre de la app. Una vez sellada, siempre tiene lo mismo.	Guardas algo que puede cambiar con el tiempo. Ejemplo: tu saldo bancario, la temperatura. Hoy tiene un valor, mañana otro.

```
// 📦 const: el nombre de tu app nunca cambia
const NOMBRE_APP = 'Mi Simulador de Finanzas';

// 💰 let: el saldo sí puede cambiar
let saldo = 0;
let ingreso = 1500; // en dólares o la moneda que prefieras
let gasto = 800;

// ❌ Esto daría ERROR — const no se puede reasignar
// NOMBRE_APP = 'Otro nombre'; // SyntaxError
```

Funciones — La receta de cocina

Una función es como una receta: la escribes UNA vez y la usas mil veces. Cambias los ingredientes (parámetros) y obtienes resultados distintos sin reescribir nada.

```
// 📖 Definir la función (escribir la receta)
function calcularSaldo(ingreso, gasto) {
  const resultado = ingreso - gasto;
  return resultado; // devuelve el plato cocinado
}

// 🍳 Usar la función (cocinar con la receta)
let miSaldo = calcularSaldo(1500, 800);
console.log('Tu saldo es: $' + miSaldo); // → Tu saldo es: $700

// La misma receta con otros ingredientes
let otroSaldo = calcularSaldo(3000, 2100);
```

```
console.log('Tu saldo es: $' + otroSaldo); // → Tu saldo es: $900
```

Recursos Visuales — Pizarra Módulo 01

Dibuja en la pizarra: "El Modelo de Cajas"

1. Dibuja 3 rectángulos grandes y etiquétalos: NOMBRE_APP, saldo, ingreso
 2. Dentro de cada rectángulo escribe el valor actual.
 3. Dibuja un candado sobre NOMBRE_APP (es const). Las otras no tienen candado.
 4. Muestra con una flecha que calcularSaldo() "lee" ingreso y gasto, hace la resta, y "produce" un nuevo número.
- Mensaje clave: "Las variables son cajas. Las funciones son máquinas que transforman cajas en resultados."

MINI-RETO 01 — 5 minutos

Instrucciones del reto

Abre la consola del navegador (F12 → Console) y escribe el siguiente código.

Luego responde: ¿Qué pasa si cambias ingreso a 500? ¿Y si gasto es mayor que ingreso?

Bonus: Agrega una variable "ahorro" que sea el 10% del ingreso.

```
// === MINI-RETO 01 ===  
// Modifica estos valores y observa el resultado  
let ingreso = 2000; // ← cambia este número  
let gasto   = 1200; // ← y este también  
  
function calcularSaldo(ing, gas) {  
  return ing - gas;  
}  
  
let saldo = calcularSaldo(ingreso, gasto);  
console.log('Saldo: $' + saldo);  
  
// 🏆 BONUS: agrega tu variable 'ahorro' aquí
```

MÓDULO 02 — Lógica y Condicionales (0:30 – 1:00)

Conceptos con Analogías

if / else — El semáforo financiero

Un if/else es exactamente como un semáforo: dependiendo del estado (condición), el coche toma un camino u otro. El código no puede hacer ambas cosas al mismo tiempo.

```
// 🚦 Semáforo financiero: verde, amarillo o rojo
function estadoFinanciero(saldo) {
  if (saldo > 500) {
    return '🟢 Excelente — estás ahorrando bien';
  } else if (saldo > 0) {
    return '🟡 Cuidado — estás en positivo pero ajustado';
  } else {
    return '🔴 Alerta — tienes más gastos que ingresos';
  }
}

console.log( estadoFinanciero(700) ); // 🟢 Excelente
console.log( estadoFinanciero(200) ); // 🟡 Cuidado
console.log( estadoFinanciero(-100) ); // 🔴 Alerta
```

Operadores — Los comparadores del árbitro

Los operadores de comparación son como un árbitro de fútbol: comparan dos jugadores (valores) y dan un resultado de true o false.

Operador	Significado	Ejemplo
===	Exactamente igual (valor y tipo)	500 === 500 → true
!==	Diferente	500 !== 300 → true
>	Mayor que	700 > 500 → true
<	Menor que	300 < 500 → true
>=	Mayor o igual	500 >= 500 → true

Recursos Visuales — Pizarra Módulo 02

Dibuja en la pizarra: "El Diagrama de Flujo del Semáforo"

1. Dibuja un rombo (decisión) con la pregunta: "¿saldo > 500?"
2. De él salen dos flechas: SÍ (verde) y NO (continúa).
3. El NO llega a otro rombo: "¿saldo > 0?"
4. Ese rombo también tiene SÍ (amarillo) y NO (rojo).

Mensaje clave: "El código piensa como tú, pero solo puede responder SÍ o NO en cada paso."

MINI-RETO 02 — 5 minutos

Instrucciones del reto

Copia la función estadoFinanciero() en la consola.

Agrega una cuarta condición: si el saldo es mayor a 2000, muestra "★ ¡Modo inversión activado!".

Bonus: Prueba qué pasa si llamas a la función con un texto en vez de un número.

```
// === MINI-RETO 02 ===
// Agrega una cuarta condición al semáforo
function estadoFinanciero(saldo) {
  if (saldo > 500) {
    return '🟢 Excelente';
  } else if (saldo > 0) {
    return '🟡 Cuidado';
  } else {
    return '🔴 Alerta';
  }
}

// 🏆 BONUS: agrega el caso saldo > 2000 ANTES del primer if

// Prueba con distintos valores
console.log( estadoFinanciero(3000) );
console.log( estadoFinanciero(0) );
```

MÓDULO 03 — El DOM y los Eventos (1:00 – 1:30)



Conceptos con Analogías

El DOM — El árbol de LEGO de tu página

El DOM (Document Object Model) es como ver tu página HTML como piezas de LEGO. JavaScript puede agarrar cualquier pieza por su nombre (id) y cambiarla, moverla o hacerla desaparecer. El HTML es el plano; el DOM es el juguete armado.

```
<!-- HTML: el 'plano' de tu página -->
<div id='saldo-display'>$0</div>
<button id='calcular-btn'>Calcular</button>
<span id='estado-emoji'>🕒</span>
```

```
// JavaScript: modificamos las 'piezas de LEGO'

// 1. Agarrar la pieza por su id
const pantallasSaldo = document.getElementById('saldo-display');

// 2. Cambiarle el contenido
pantallasSaldo.innerHTML = '$700';

// 3. Cambiarle el estilo
pantallasSaldo.style.color = 'green';
```

Eventos — El timbre de la puerta

Un evento es como el timbre de tu casa: alguien lo toca (hace clic, escribe algo, mueve el mouse) y tu código reacciona. Tú decides qué hacer cuando suena el timbre.

```
// 📞 'Escuchar' el timbre del botón
const boton = document.getElementById('calcular-btn');

boton.addEventListener('click', function() {
  // Esto corre SOLO cuando el usuario hace clic
  console.log('¡El timbre sonó! El usuario hizo clic.');
```



```
  // Aquí va toda la lógica del simulador
  actualizarDashboard();
});
```

Leyendo valores del usuario — El traductor Number()

ADVERTENCIA IMPORTANTE: Todo lo que el usuario escribe en un input llega como texto, aunque escriba un número. Number() es el traductor que convierte "1500" (texto) a 1500 (número real).

```
const inputIngreso = document.getElementById('input-ingreso');

// ❌ MAL: valor llega como TEXTO '1500'
let valorMal = inputIngreso.value;           // '1500' (string)
console.log(valorMal + 200);                  // '1500200' - concatena!

// ✅ BIEN: convertir a número primero
let valorBien = Number(inputIngreso.value);  // 1500 (number)
console.log(valorBien + 200);                 // 1700 - suma correctamente
```

Recursos Visuales — Pizarra Módulo 03

Dibuja en la pizarra: "El Flujo de Datos Usuario → DOM"

1. Dibuja al usuario escribiendo en un <input>.
 2. Una flecha: input.value → Number() → variable JS.
 3. La variable entra a la función calcularSaldo().
 4. El resultado sale de la función → innerHTML actualiza el <div>.
 5. El usuario ve el nuevo número en pantalla.
- Mensaje clave: "El código conecta lo que el usuario escribe con lo que ve en pantalla."

MINI-RETO 03 — 5 minutos

Instrucciones del reto

Crea un archivo HTML con el código de abajo.

Haz que al hacer clic el botón cambie el color del texto del saldo: verde si es positivo, rojo si es negativo.

Bonus: Agrega un segundo botón "Resetear" que limpie todos los valores a cero.

```
<!-- === MINI-RETO 03 === -->
<!DOCTYPE html>
<html>
<body>
  <input id='inp-ingreso' type='number' placeholder='Ingreso'>
  <input id='inp-gasto' type='number' placeholder='Gasto'>
  <button id='btn-calc'>Calcular</button>
  <p>Saldo: <span id='resultado'>$0</span></p>
```



```
<script>
  document.getElementById('btn-calc').addEventListener('click', function() {
    let ing = Number(document.getElementById('inp-ingreso').value);
    let gas = Number(document.getElementById('inp-gasto').value);
    let sald = ing - gas;
    document.getElementById('resultado').innerHTML = '$' + sald;
    // 🏆 BONUS: cambia el color según el valor de sald
  });
</script>
</body>
</html>
```

MÓDULO 04 — Proyecto Final Completo (1:30 – 2:00)



Código Completo del Simulador

Aquí está la app entera. Cada parte fue aprendida en los módulos anteriores. Ahora las unimos.

```
<!DOCTYPE html>
<html lang='es'>
<head>
  <meta charset='UTF-8'>
  <title>Simulador de Finanzas Personales</title>
  <style>
    body { font-family: sans-serif; max-width: 480px;
      margin: 40px auto; padding: 20px; background: #f0f4f8; }
    h1 { color: #1c2833; text-align: center; }
    .card { background: white; border-radius: 12px;
      padding: 20px; margin-bottom: 16px;
      box-shadow: 0 2px 8px rgba(0,0,0,0.08); }
    input { width: 100%; padding: 10px; margin-top: 8px;
      border: 1px solid #ccc; border-radius: 8px;
      font-size: 16px; box-sizing: border-box; }
    button { width: 100%; padding: 14px; margin-top: 16px;
      background: #1a73e8; color: white; border: none;
      border-radius: 8px; font-size: 16px; cursor: pointer; }
    button:hover { background: #1558b0; }
    #panel-resultado { display: none; } /* oculto al inicio */
    #saldo-display { font-size: 42px; font-weight: bold;
      text-align: center; margin: 10px 0; }
    #estado-texto { text-align: center; font-size: 18px; }
  </style>
</head>
<body>
  <h1>💰 Mi Simulador de Finanzas</h1>

  <!-- PANEL DE INPUTS -->
  <div class='card'>
    <label>Ingreso mensual ($)</label>
    <input id='inp-ingreso' type='number' placeholder='Ej: 2000'>
    <label>Gastos mensuales ($)</label>
    <input id='inp-gasto' type='number' placeholder='Ej: 1200'>
    <button id='btn-calcular'>Calcular mi saldo ▶</button>
  </div>

  <!-- PANEL DE RESULTADO (se muestra al calcular) -->
  <div class='card' id='panel-resultado'>
    <p id='estado-texto'></p>
    <div id='saldo-display'></div>
    <p id='consejo-texto'></p>
  </div>
```

```

<!-- SCRIPT: al final del body para que el HTML ya exista -->
<script>
  // — Módulo 1: constantes y función —————
  const META_AHORRO = 500; // umbral para 'modo inversión'

  function calcularSaldo(ingreso, gasto) {
    return ingreso - gasto;
  }

  // — Módulo 2: lógica del semáforo —————
  function obtenerEstado(saldo) {
    if (saldo > META_AHORRO * 4) return { emoji: '★', msg: '¡Modo inversión!',
color: '#8E44AD', consejo: 'Considera invertir tu excedente.' };
    if (saldo > META_AHORRO) return { emoji: '🟢', msg: 'Excelente salud',
color: '#27AE60', consejo: 'Vas muy bien. Mantén ese ritmo.' };
    if (saldo > 0) return { emoji: '🟡', msg: 'Cuidado, ajustado',
color: '#E67E22', consejo: 'Revisa qué gastos puedes reducir.' };
    return { emoji: '🔴', msg: 'Gastos exceden
ingresos', color: '#E74C3C', consejo: 'Urgente: reduce gastos o busca ingresos
extra.' };
  }

  // — Módulo 3: eventos y DOM —————
  document.getElementById('btn-calcular').addEventListener('click', function() {
    // Leer y convertir valores
    const ingreso = Number(document.getElementById('inp-ingreso').value);
    const gasto = Number(document.getElementById('inp-gasto').value);

    // Validación básica
    if (ingreso === 0 && gasto === 0) {
      alert('Por favor ingresa al menos un valor.');
```



Ideas para extender el proyecto (siguientes pasos)

Agregar una lista de gastos con `Array.push()` y mostrarlos con un bucle `forEach`.

Usar `localStorage` para que los datos persistan al recargar la página.

Agregar una gráfica de "Ingresos vs Gastos" con la librería `Chart.js`.

Calcular el porcentaje de ahorro: $(\text{saldo} / \text{ingreso} * 100).toFixed(1) + \%$



"¿Por qué no funciona mi código?" — Guía de Errores Comunes

Esta sección es tu salvavidas. Guárdala. Estos son los 6 errores que el 95% de principiantes cometen en sus primeras horas.

✗ Error 1 — ID mal escrito (el error #1 en el mundo)

```
<!-- HTML -->
<div id='saldo-display'>$0</div>

// JavaScript — ¿ves el error? El id tiene un guión extra
const panel = document.getElementById('saldo--display'); // ✗
panel.innerHTML = '$700'; // ERROR: Cannot set properties of null

// ✔ Solución: copiar el id EXACTO del HTML, incluyendo mayúsculas
const panel = document.getElementById('saldo-display'); // ✔
```

Diagnóstico

El mensaje "Cannot set properties of null" casi siempre significa que `getElementById()` no encontró el elemento.

Solución: Ve al HTML, copia el id con Ctrl+C y pégalo en el JS. Nunca lo escribas a mano.

✗ Error 2 — Olvidar `Number()` (el más frustrante)

```
// El usuario escribe '1500' en el input
const valor = document.getElementById('inp-ingreso').value;

// ✗ Resultado inesperado: concatena en vez de sumar
console.log(valor + 200); // '1500200' ← ¡Es un texto pegado!

// ✔ Solución: convertir SIEMPRE con Number()
const valorNum = Number(valor);
console.log(valorNum + 200); // 1700 ← ahora sí suma
```

Diagnóstico

Si ves números "pegados" (como 1500200) en vez de sumas, olvidaste `Number()`.

Regla de oro: cualquier valor que venga de un `<input>` SIEMPRE pasa por `Number()` antes de usarlo en matemáticas.

✗ Error 3 — El script está en el `<head>` (el HTML no existe aún)

```

<!-- ❌ MAL: el script intenta leer elementos que AÚN no existen -->
<head>
  <script>
    const btn = document.getElementById('mi-boton'); // → null
  </script>
</head>
<body>
  <button id='mi-boton'>Clic</button> <!-- llega tarde -->
</body>

<!-- ✅ BIEN: el script va AL FINAL del <body> -->
<body>
  <button id='mi-boton'>Clic</button>
  <script>
    const btn = document.getElementById('mi-boton'); // ✅ ya existe
  </script>
</body>

```

Diagnóstico

El navegador lee el HTML de arriba hacia abajo. Si el script va antes que el elemento, el elemento no existe cuando JS lo busca.

Solución permanente: pon SIEMPRE el <script> como último hijo de <body>.

❌ Error 4 — Usar = en vez de === para comparar

```

let saldo = 0;

// ❌ MAL: un solo = ASIGNA, no compara
if (saldo = 500) { // esto pone 500 en saldo — siempre verdadero!
  console.log('Esto siempre corre aunque saldo sea 0');
}

// ✅ BIEN: tres iguales COMPARAN valor y tipo
if (saldo === 500) {
  console.log('Solo corre si saldo es exactamente 500');
}

```

Diagnóstico

= es el operador de asignación (guarda un valor en una variable).

=== es el operador de comparación (pregunta si dos cosas son iguales).

Si tu if siempre se ejecuta sin importar los valores, probablemente usaste = en vez de ===.

❌ Error 5 — Olvidar las comillas en los IDs

```
// ❌ MAL: sin comillas, JS cree que 'saldo-display' es una variable
const panel = document.getElementById(saldo-display); // ReferenceError

// ✅ BIEN: el id siempre entre comillas (simples o dobles)
const panel = document.getElementById('saldo-display');
const panel = document.getElementById("saldo-display"); // también válido
```

❌ Error 6 — innerHTML vs innerText vs value (¿cuál usar?)

```
// innerHTML → para cambiar contenido HTML de un elemento
document.getElementById('resultado').innerHTML = '<strong>$700</strong>';

// innerText → para cambiar solo texto (sin HTML)
document.getElementById('resultado').innerText = '$700';

// .value → SOLO para leer/escribir en <input>, <select>, <textarea>
document.getElementById('inp-ingreso').value = '1500';

// Error común: usar .value en un <div> o <p>
document.getElementById('resultado').value = '$700'; // ❌ No funciona en divs
```

Síntoma	Causa probable	Solución
Cannot set properties of null	ID mal escrito o script en <head>	
Números se "pegan" (1500200)	Falta Number()	
El if siempre corre	= en vez de ===	
ReferenceError inesperado	Falta comillas en getElementById	
Nada cambia en pantalla	innerHTML en un <input>	

🎓 ¡Lo lograste! — Resumen de lo aprendido hoy

Concepto aprendido	Lo usaste para...
const / let	Guardar ingreso, gasto y saldo
Funciones	calcularSaldo() y obtenerEstado()
Condicionales if/else	El semáforo financiero (verde/amarillo/rojo)
DOM — getElementById	Encontrar elementos HTML desde JS

DOM — innerHTML / style	Mostrar el saldo y cambiar colores
Eventos — addEventListener	Reaccionar al clic del botón
Number()	Convertir texto de inputs a números reales

Próximos pasos sugeridos: Arrays y bucles (forEach) → Objetos JS → fetch() para consumir una API de tasas de cambio en tiempo real.